



MANUAL TEST GUIDE · MODULE 03 OF 11

Sales & CRM

Functional-Flow Test Walkthroughs

Scenario-driven, hands-on QA guide for the **logistics.sales_crm** commercial control layer — the 14-step lifecycle from inbound inquiry through lead, opportunity, priced quote, pricing approval, customer acceptance, and the auto-created operations **handover**. "No re-keying" is the delivery standard every step verifies.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives you a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

01 What Sales & CRM does & how to read this guide

Sales & CRM is the **commercial control layer**: it captures customer intent, qualifies it, prices it correctly, enforces pricing governance, and hands accepted business to operations — without manual re-keying. Five transactional records carry one deal: `crm.inquiry` → `crm.lead` → `crm.opportunity` → `crm.quote` (with a versioned `crm.quote.version`) → `crm.handover`. Each conversion copies party + routing + commercial context forward.

38

DOMAIN COMMANDS

3

TEST SCENARIOS

5

LIFECYCLE STATE MACHINES

10

QUOTE STAGES

The five state machines you will observe

Inquiry / lead / opportunity / quote status fields are `workflow=True` — direct writes raise `StageWriteForbidden`; transitions flow through `env.workflow.transition_by_command`. Handover is a plain command-driven state field (8 states).

```
inquiry : NEW_PROSPECT → PROSPECT_DISQUALIFIED
lead    : LEAD_QUALIFIED → LEAD_DROPPED
opp     : OPP_PROPOSAL_SENT → OPP_NEGOTIATION → OPP_WON | OPP_LOST
quote   : QUOTE_DRAFT → SUBMITTED → UNDER_APPROVAL → APPROVED → SENT → ACCEPTED | REJECTED | EXPIRED | CANCELLED | LOST
handover : draft → submitted → under_review → accepted → completed (↳ returned / escalated / rejected)
```

PRICING-APPROVAL ROUTING

- **low-margin** — `margin_below_threshold==True` (mode×lan×tier floor)
- **sell-below-buy** — `total_sell_amount < total_cost_amount`
- Payment-term / service-scope / validity-extension rules also route
- Case lands on `sales_crm.sales_manager` role-bound users

MULTI-LEG ROUTING (QUOTE.ROUTE)

- One quote version, N `crm.quote.route` legs
- `leg_type` : origin_haulage · main_carriage · transshipment · destination_haulage
- Door-to-door with transshipment = 4 legs, one deal
- Transit days roll up to the version; route summary flows to the handover

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Full commercial pipeline	Globex happy path: inquiry → won opportunity → priced quote → approval → accept → handover auto-created, all carry-forward (no re-keying)	<code>convert_to_lead</code> , <code>create_quote</code> , <code>accept</code>	\$02 · \$03
B — Approval & negotiation	Low-margin / sell-below-buy approval routing, the no-send gate, spot-rate RFQ, negotiation log & quote revision	<code>request_approval</code> , <code>log_negotiation</code> , <code>revise</code>	\$04
C — Handover & lost states	Handover ops lifecycle, return/resubmit rework guard, Y-fork routing, plus lead-drop / opp-lost / inquiry-disqualify storylines	<code>submit</code> , <code>route_to_fulfillment</code> , <code>drop</code>	\$05

Demo fixtures you start from (loaded by `--with-demo=all`)

Forwarder **Acme Forwarding** — **Mumbai**; customers **Globex** (Platinum), **Stark** (Gold), **Wayne** (Silver). Globex ships an inquiry → lead → opportunity → draft quote chain (`demo_quote_globex_001` , `QUOTE_DRAFT`); Stark sits mid-flight at `QUOTE_SENT` . **Workflow note**: XML seeds records only in their *initial* stage — every advance (Submit → Send → Accept, Drop, Disqualify, Close Lost) is a browser click, which is the demo.

02 Full pipeline — inquiry to a priced, won quote

Goal: carry the Globex enquiry from an inbound inquiry to a won opportunity with a priced draft quote — every field copied forward, nothing re-typed. **Role:** Sales Rep. **Start state:** `demo_inquiry_globex_001` in `NEW_PROSPECT`, not yet converted.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open the inquiry; run Assign (<code>crm.inquiry.assign</code>) with a <code>user_id</code> .	Inquiry <code>ownership_user_id</code> is updated to the chosen rep.	Owner reflects the assignee.	command
2	Run Convert to Lead (<code>crm.inquiry.convert_to_lead</code>).	A <code>crm.lead</code> is created in LEAD_QUALIFIED ; partner, contact, service type, origin/destination, shipment type, transport mode, load type & service mode are copied . Inquiry <code>converted_lead_id</code> set → button hides.	Lead carries every routing field; navigates to it.	lifecycle
3	On the lead, run Convert to Opportunity (<code>crm.lead.convert_to_opportunity</code>).	A <code>crm.opportunity</code> is created in OPP_PROPOSAL_SENT with carried-forward context + estimated revenue. Lead <code>converted_opportunity_id</code> set.	Opportunity created; button hides.	lifecycle
4	(Optional) Start Negotiation (<code>crm.opportunity.start_negotiation</code>), then Close Won (<code>crm.opportunity.close_as_won</code>).	Stage walks to OPP_NEGOTIATION then OPP_WON . Guard <code>opp_has_close_date</code> passes (a default close date is stamped at create).	Status = OPP_WON.	guard: <code>opp_has_close_date</code>
5	Run Create Quote (<code>crm.opportunity.create_quote</code>).	A draft <code>crm.quote</code> (QUOTE_DRAFT) is created; the <code>post.crm.quote.create</code> hook auto-mints v1 and points <code>current_version_id</code> at it. Issue date today, expiry +30 days, payment terms Net 30.	Quote + v1 exist; header pre-filled from the opportunity.	hook: <code>post.create</code>
6	On v1, run the rate picker — <code>crm.quote.create_lines_from_rate</code> with an approved <code>pricing.rate</code> .	Each <code>pricing.rate.line</code> is copied as a <code>crm.quote.line</code> with a frozen buy/sell snapshot (<code>buy_snapshot</code> / <code>sell_snapshot</code> , <code>source_rate_id</code>) — immutable even if the source rate later changes.	Lines present; snapshot amounts stored on each line.	rate picker

Multi-leg emphasis — one quote, many legs

For a door-to-door move (e.g. Wayne Mumbai → Singapore T/S → Sydney) author **four** `crm.quote.route` rows on the single version: `origin_haulage` → `main_carriage` → `transshipment` → `destination_haulage`. Confirm the legs order by `leg_sequence`, transit days roll up to the version, and the composed `route_summary` is the same string that later lands in the handover's `special_instruction_text`. The deal stays **one** quote — never one record per leg.

⚠ The rate snapshot is a freeze, not a link

Editing the source `pricing.rate` after the picker runs must **not** change the quote line — the buy/sell snapshot is what the customer is quoted. Submit (\$03) re-validates the source rate is still **approved** and unexpired via `validate_source_rate_usable`; a suspended rate blocks submit.

03 Quote lifecycle to accepted & the auto-handover

Goal: drive the priced quote through submit → approval → send → accept, and confirm the **handover materializes automatically** on accept with denormalized commercial context. **Start state:** the QUOTE_DRAFT from \$02 with ≥1 charge line.

#	Action	Expected result	✓ Checkpoint	Trace
7	Run Submit (<code>crm.quote.submit</code>).	Requires <code>current_version_id</code> + ≥1 charge line; re-validates each line's source rate. Stage → QUOTE_SUBMITTED .	Status = QUOTE_SUBMITTED.	gate
8	Run Request Approval (<code>crm.quote.request_approval</code>).	The <code>mode×lane×tier</code> margin floor is resolved and stamped (<code>margin_below_threshold</code>); an <code>ir.approval.case</code> opens. Stage → QUOTE_UNDER_APPROVAL .	Status = UNDER_APPROVAL; a case exists.	WS4
9	In the Approvals inbox (admin bound to <code>sales_crm.sales_manager</code>), Approve the case.	The event bridge advances the quote UNDER_APPROVAL → QUOTE_APPROVED (async — poll the DB, not the DOM).	Status = QUOTE_APPROVED.	approval bridge
10	Run Send to Customer (<code>crm.quote.send</code>).	No-send gate passes (no open approval case remains). Stage → QUOTE_SENT ; <code>accepted_at</code> slot prepared.	Status = QUOTE_SENT.	WS5 no-send gate
11	Run Customer Accepted (<code>crm.quote.accept</code>) — quote header carries a <code>contact_id</code> .	Guard <code>quote_not_expired</code> passes; pre-accept revalidation (FX / service-scope / buying-rate) passes; a <code>crm.quote.acceptance</code> row (<code>decision_type=accepted</code>) is written; stage → QUOTE_ACCEPTED ; <code>accepted_at</code> stamped.	Status = ACCEPTED; acceptance row present.	guard: quote_not_expired · QAB-06
12	Open the Handovers menu.	A <code>crm.handover</code> was auto-created in draft (<code>crm.handover.auto_create_on_accept</code> default true) with denormalized partner / contact / service-type / incoterm / currency / totals, seeded customer + shipper <code>crm.handover.party</code> rows, a policy-driven document checklist, and a cargo summary.	Exactly one handover references this quote.	Enh 06

✓ Scenario A pass criteria

The deal travelled inquiry → lead → opportunity → quote → handover with **zero re-keying** — every conversion copied context forward, the priced lines froze a rate snapshot, the approval routed and cleared, and the handover materialized on accept without a manual create.

⚠ Subject-lock timing

Add all quote children (lines / routes / packages) while the version is **draft**. Once the version reaches `sent` / `accepted` / `superseded` the subject-lock freezes it — the only escape is `crm.quote.revise` , which mints a fresh `version_number+1` and marks the prior `superseded` (\$04).

04 Pricing approval, spot rates & negotiation

Goal: exercise the commercial-governance surface — the approval rules that must route a thin-margin quote, the gate that stops a quote leaking to the customer mid-approval, spot-rate RFQs, and the negotiation-log → revision loop. **Role:** Sales Rep + Pricing Approver.

#	Action	Expected result	✓ Checkpoint	Trace
1	Price a quote below the resolved margin floor (Globex = Platinum, SEA lane floor 6%), then Request Approval .	The <code>low_margin</code> rule fires (<code>margin_below_threshold==True</code>) and routes the case up the pricing chain — sales manager then pricing approver.	Low-margin chain selected, not the default.	WS4 low_margin
2	Price a line so <code>total_sell_amount < total_cost_amount</code> , then Request Approval .	The <code>sell_below_buy</code> rule fires and escalates to the mandatory finance / country-head chain.	Sell-below-buy escalation selected.	WS4 sell_below_buy
3	While the case is still open, attempt Send (<code>crm.quote.send</code>).	Refused — "cannot send to the customer while an approval is still open". The quote cannot bypass the approval chain.	Send blocked mid-approval.	WS5 no-send gate
4	On the opportunity, run Request Spot Rate (<code>crm.opportunity.request_spot_rate</code>).	A <code>pricing.spot.request</code> is raised carrying <code>related_model="crm.opportunity"</code> + <code>related_id</code> ; route / commodity / customer default from the opportunity. Surfaces read-only under <code>spot_request_ids</code> .	Spot RFQ linked back to the opportunity.	Phase 2 WS3
5	On a sent quote, Log Negotiation (<code>crm.quote.log_negotiation</code>) — e.g. <code>negotiation_type="price"</code> + a customer request text.	A <code>crm.quote.negotiation</code> row is appended with <code>outcome="pending"</code> . Type must be one of price / validity_extension / cargo_change / routing_change / service_addition / payment_term / other.	Pending negotiation logged.	QAB-03
6	Run Revise (<code>crm.quote.revise</code>) — optionally with <code>from_negotiation_id</code> .	A new version (<code>version_number+1</code> , status draft) is minted; lines / routes / packages copied; prior version marked superseded ; <code>current_version_id</code> re-pointed. A linked pending negotiation auto-resolves to <code>accepted_into_revision</code> .	New editable version; prior frozen.	QAB-03
7	Resolve a standalone negotiation via Resolve Negotiation (<code>crm.quote.resolve_negotiation</code>) with <code>outcome</code> ∈ {accepted_into_revision, declined, withdrawn}.	Outcome stamped; <code>revision_version_id</code> required when <code>accepted_into_revision</code> . Re-resolving a resolved row is refused.	Outcome recorded once.	QAB-03
8	Terminal branches: Reject (<code>crm.quote.reject</code> , needs <code>lost_reason_id</code>), Mark Lost (<code>crm.quote.mark_lost</code>), Mark Expired (<code>crm.quote.mark_expired</code>), Cancel (<code>crm.quote.cancel</code>).	Stage lands in QUOTE_REJECTED / LOST / EXPIRED / CANCELLED . Reject writes a <code>crm.quote.acceptance</code> (<code>decision_type=rejected</code>) — no handover is created.	Terminal reached; reject creates no handover.	guard: <code>quote_has_lost_reason</code>

✓ Scenario B pass criteria

Thin-margin and sell-below-buy quotes both route to approval rather than sending directly; the no-send gate holds while a case is open; spot RFQs bind back to the opportunity; and the negotiation → revise loop supersedes the prior version while keeping it immutable for audit.

05 Handover ops lifecycle, Y-fork routing & lost states

Goal: drive the auto-created handover through the ops review lifecycle, prove the rework guardrail and the frozen Y-fork routing slot, then walk the three lost / dropped storylines. **Role:** Ops Manager (review) + Sales Rep (corrections). **Start:** the **draft** handover from §03.

#	Action	Expected result	✓ Checkpoint	Trace
1	On the handover, resolve the required doc-items, add a consignee party, then Submit (<code>crm.handover.submit</code>).	Submit guards pass (shipper + consignee present; no required doc pending). Status draft → submitted .	Status = submitted.	Enh 06 guards
2	Start Review (<code>crm.handover.start_review</code>), then Return to Sales (<code>crm.handover.return_to_sales</code>) with a <code>reason_text</code> .	Status → under_review → returned ; <code>return_reason_text</code> stored; a doc-related reason resets <code>document_check_status</code> to pending. Opportunity chatter mirrors the return.	Returned with reason; opp notified.	QAB-11
3	Sales fixes, runs Confirm Documents (<code>crm.handover.confirm_documents</code>), then Resubmit (<code>crm.handover.resubmit_after_return</code>).	<code>document_check_status=confirmed</code> ; status returned → submitted ; <code>return_count</code> increments. After <code>crm.handover.auto_reject_after_returns</code> (default 3) the next resubmit auto-rejects.	Resubmitted; return_count +1.	QAB-20
4	Confirm documents, then Accept Handover (<code>crm.handover.accept</code>).	Requires <code>document_check_status=confirmed</code> . Status → accepted ; <code>acknowledged_at</code> stamped; <code>crm.handover.accepted</code> event fires.	Status = accepted.	QAB-11
5	Route to Fulfillment (<code>crm.handover.route_to_fulfillment</code>) with a downstream <code>target_code</code> (e.g. booking).	Neutral slot <code>fulfillment_target_id</code> set once; <code>fulfillment_status=routed</code> ; <code>crm.handover.path_decided</code> event fires. Re-routing to a different target is frozen.	Routed exactly once.	BR-HO-ROUTE-01
6	Consumer advances Set Fulfillment Status (<code>crm.handover.set_fulfillment_status</code>) → <code>in_fulfillment</code> → fulfilled .	On <code>fulfilled</code> the producer self-completes: <code>handover_status</code> → <code>completed</code> , <code>completed_at</code> stamped, <code>crm.handover.fulfillment_completed</code> + <code>completed</code> events fire.	Handover auto-completes on fulfilled.	Enh 08

LOST / DROPPED STORYLINES (EACH A TERMINAL STATE)

- **Lead drop** — `crm.lead.drop` (optional `drop_reason_id`) → **LEAD_DROPPED**. (Wayne went dark after follow-ups.)
- **Opportunity lost** — `crm.opportunity.close_as_lost` requires `lost_reason_id` (guard `opp_lost_has_reason`) → **OPP_LOST**. (Stark lost to competitor.)
- **Inquiry disqualify** — `crm.inquiry.disqualify` → **PROSPECT_DISQUALIFIED** (unsupported hazardous lane).

✓ Scenario C pass criteria

The handover walks `draft` → `submitted` → `under_review` → `returned` → `resubmitted` → `accepted` → `completed` with the rework guard enforced, the Y-fork slot frozen after one route, and every lost storyline landing in its structured terminal state with a captured reason.

06 The guards that must refuse — real error messages

Half of testing a commercial spine is confirming it **refuses** bad input. Each row below trips a specific guard and states the **real** error string the command raises. The command must reject — no partial write, no state advance.

#	Action	Expected refusal (real message)	✓
1	<code>convert_to_opportunity</code> on a lead not in LEAD_QUALIFIED.	"crm.lead.convert_to_opportunity: lead must be in LEAD_QUALIFIED status".	Blocked
2	<code>create_quote</code> on an opportunity with no customer.	"...opportunity must have a customer (partner_id) before a quote can be drafted from it."	Blocked
3	<code>crm.quote.submit</code> on a quote with no current version.	"...quote has no current version — create one first via <code>crm.quote.revise</code> or initial save."	Blocked
4	<code>crm.quote.submit</code> on a version with no charge lines.	"...the current version has no charge lines — add at least one before submitting."	Blocked
5	<code>crm.quote.accept</code> with no contact on payload or header.	"...customer-side contact is required to capture acceptance (decided_by_contact_id on payload, or contact_id on the quote header)..."	Blocked
6	<code>crm.quote.reject</code> without a <code>lost_reason_id</code> .	"...lost_reason_id is required so the rejection feeds win/loss analytics. Pass it on the command payload."	Blocked
7	<code>crm.quote.send</code> while an approval case is still open.	"...cannot send to the customer while an approval is still open (case state=...)..."	Blocked
8	<code>create_lines_from_rate</code> from a non-approved rate.	"Rate <n> is not approved (status=...) — cannot quote it."	Blocked
9	<code>crm.handover.submit</code> missing shipper / consignee.	"crm.handover.submit: missing required parties — consignee. Add shipper/consignee party rows before submitting..."	Blocked
10	<code>crm.handover.accept</code> before documents confirmed.	"...document_check_status must be 'confirmed' before accept — run <code>crm.handover.confirm_documents</code> first."	Blocked
11	<code>route_to_fulfillment</code> on a handover not yet accepted.	"...handover_status must be 'accepted' to route; got ..."	Blocked
12	<code>route_to_fulfillment</code> a second time (already routed).	"crm.handover is already routed ... cannot re-route (BR-HO-ROUTE-01) — clear the route first."	Blocked
13	<code>crm.handover.return_to_sales</code> without a <code>reason_text</code> .	"crm.handover.return_to_sales: reason_text is required."	Blocked
14	<code>crm.quote.log_negotiation</code> with no customer request text.	"crm.quote.log_negotiation: customer_request_text is required."	Blocked

⚠ Workflow vs. command guards

Status-field refusals differ by layer. A **direct write** to a `workflow=True` status raises `StageWriteForbidden` from the kernel; a **workflow guard** (`opp_has_close_date`, `opp_lost_has_reason`, `quote_not_expired`, `quote_has_lost_reason`) raises `GuardFailed` from the engine; the messages above are the **command-handler** `ValueError`s that run before or instead of a transition. Confirm each surfaces as an error toast, not a silent no-op.

✓ Full-module pass criteria

All three scenarios green — pipeline carry-forward + auto-handover (A), approval routing + no-send gate + negotiation loop (B), handover ops lifecycle + Y-fork + lost states (C) — and every negative guard in §06 confirmed to refuse with its real message.

Traceability: QAB- IDs map to the Sales-CRM BRS (QAB-03 negotiation, QAB-06 accept revalidation, QAB-11 handover states, QAB-20 rework rate); BR-HO-ROUTE-01 to the handover Y-fork mutual-exclusion invariant; WS4/WS5 to the Phase 2 pricing-approval work streams. Cross-module effects (booking created off the handover) are verified in the Booking module's own test guide.*